



Section 11 / 12

LLM-based Prompt Injection Mitigations

As we have seen in the previous section, traditional mitigations are typically inadequate to protect against prompt injection attacks. Therefore, we will explore more sophisticated mitigations in this section. These include mitigations trained into the LLM during model training, as well as using a separate guardrail LLM to detect and block prompt injection attacks.

Fine-Tuning Models

When deploying an LLM for any purpose, it is generally good practice to consider what model best fits the required needs. There is a wide variety of open-source models out there. Choosing the right one can significantly impact the quality of the generated responses and resilience against prompt injection attacks.

To further increase robustness against prompt injection attacks, a pre-existing model can be **fine-tuned to the specific use case through additional training**. For instance, a tech support chatbot could be trained on a dataset of tech support chat logs. While this does not eliminate the risks of successful prompt injection attacks, it narrows the scope of operation and thus reduces the LLM's susceptibility to prompt injection attacks. Additionally, a fine-tuned model might generate responses of higher quality due to the specialized training it received. As such, it is good practice to fine-tune a model from a quality and a security perspective.

Adversarial Prompt Training

Adversarial Prompt Training is one of the most effective mitigations against prompt injections. In this type of training, **the LLM is trained on adversarial**

prompts , including typical prompt injection and jailbreak prompts. This results in a more robust and resilient LLM, as it can detect and reject malicious prompts. The idea is that a deployed LLM exposed to a prompt injection payload will be able to react accordingly due to its training on similar prompts, providing sufficient resilience to make a successful prompt injection much more complex and time-consuming.

Many open-source LLMs, such as Meta's LLama or Google's Gemma, undergo adversarial prompt training in their regular training process. That is why these models, in their latest iterations, already offer a significantly higher level of resilience compared to their initial versions. As such, we do not necessarily need to put an LLM through adversarial prompt training ourselves if we want to deploy an open-source model.

Real-Time Detection Models

Another very effective mitigation against prompt injection is the usage of an additional **guardrail LLM** . Depending on which data they operate on, there are two kinds of guardrail LLMs: **input guards** and **output guards** .

Input guards operate on the user prompt before it is fed to the main LLM and are tasked with deciding whether the user input is malicious (i.e., contains a prompt injection payload). If the input guard classifies the input as malicious, the user prompt is not fed to the main LLM, and an error may be returned. If the input is benign, it is fed to the main LLM, and the response is returned to the user.

On the other hand, output guards operate on the response generated by the main LLM. They can scan the output for malicious or harmful content, misinformation, or evidence of a successful prompt injection exploitation. The backend application can then react accordingly and either return the LLM response to the user or withhold it, displaying an error message instead.

